ChatGPT

FancyRAG Ingestion & Knowledge Graph Build Workflows Inventory

Ingestion Entrypoints

FancyRAG currently supports two entrypoints for launching the knowledge-graph ingestion pipeline, one via a YAML-driven pipeline runner and another via an in-code orchestrator:

Entrypoint Type	Path	Symbol/Command	How Invoked
YAML Pipeline (Legacy)	`tools/run_pipeline.py`	`main()` (async)	Called by `make ingest` (uses `neo4j_graphrag.experimental.pip` to run `pipelines/kg_ingest.yaml`)
Code Pipeline (Current)	`scripts/kg_build.py` (calls `fancyrag.cli.kg_build_main`)	`kg_build_main.main()`	Run manually or via future Makefile targ `fancyrag.kg.pipeline.run_pipeli`

Canonical Path: According to project docs, the code-based pipeline (`kg_build.py`) is intended to become the canonical ingestion path, replacing the older YAML/PipelineRunner method [5]. The Makefile's current `ingest` target still uses the YAML approach, but an issue notes this should be updated to use `kg_build.py` for consistency with newer defaults [6] [7]. In summary, **the code-driven pipeline is the primary path moving forward**, while the YAML/PipelineRunner method is considered legacy.

Exhaustive Symbol Inventory

The tables below enumerate the key classes, functions, and config keys involved in FancyRAG's ingestion and KG build workflows. Each entry includes its location, signature, responsibilities, side effects, and references to callers/callees. (YAML config keys are included where they interface with code.)

Core Pipeline Orchestration (fancyrag.kg.pipeline & .phases)

Module:Symbol	Signature (or Key)	Responsibility / Description	Sic
PipelineOptions (`kg.pipeline`)	class PipelineOptions(source: Path, ..., reset_database: bool)	Data class aggregating all pipeline run inputs (source file/dir, chunking params, DB name, QA limits, semantic flags) [8] [9].	Im pa

Module:Symbol	Signature (or Key)	Responsibility / Description	Sid
run_pipeline (`kg.pipeline`)	def run_pipeline(options: PipelineOptions) -> dict[str, Any]	High-level orchestrator for an ingestion run. Resolves settings, discovers sources, builds clients, loops over sources to ingest each, then runs QA. Returns a structured run log (JSON-able dict) [14] [15].	Wr `op` pri Ne
resolve_settings (`kg.phases`)	def resolve_settings(*, profile:str|None, chunk_size:int|None, ...) -> ResolvedSettings	Merge explicit chunking overrides with profile presets. Chooses an effective chunk size, overlap, include-glob patterns, and semantic concurrency limit [21] [22]. Validates that sizes are positive/non-negative. Returns a `ResolvedSettings` record with finalized values [23] [24].	—
PROFILE_PRESETS (`kg.pipeline`)	dict PROFILE_PRESETS = {"text": {...}, "markdown": {...}, "code": {...}}	Built-in default chunking profiles. E.g. `"text"` profile = 600 size, 100 overlap, include `**/*.txt,*.md,...` [27]. Used to quickly apply preset chunk params.	—
discover_sources (`kg.phases`)	def discover_sources(*, source: Path, source_dir: Path|None, include_patterns: tuple[str,...], ...) -> SourceDiscoveryResult	Determine input sources: either a single file or all files in a directory matching patterns. Reads file(s) from disk and computes checksums [30] [31]. Returns `SourceDiscoveryResult` with a list of `SourceSpec` objects (each containing file path, relative path, text content, checksum) and flags indicating mode ("file" vs "directory") [32] [33].	Ma dir ma
SourceSpec (`kg.pipeline`)	@dataclass SourceSpec(path: Path, relative_path: str, text: str, checksum: str)	Represents a single input document to ingest, including content and provenance info [40].	—

Module:Symbol	Signature (or Key)	Responsibility / Description	Sid
build_clients (`kg.phases`)	def build_clients(*, settings: Any, chunk_size:int, chunk_overlap:int, ...) -> ClientBundle	Construct the OpenAI API clients and text splitter for this run. Creates a `SharedOpenAIClient` for API calls, then a `SharedOpenAIEmbedder` (embedding model) and `SharedOpenAILLM` (chat model) using that client, a `StructuredSemanticLLM` (for structured output) if semantic enabled, and a `CachingFixedSizeSplitter` for chunking [42] [43]. Returns a `ClientBundle` with all these components [42] [44].	Ini e.g `Sh` (re Th ch
ClientBundle (`kg.phases`)	@dataclass ClientBundle(shared_client, embedder, llm, semantic_llm, splitter)	Simple container for the client objects created by `build_clients` [52] [53]. Ensures these related components can be passed together through pipeline functions.	—

Module:Symbol	Signature (or Key)	Responsibility / Description	Sid
ingest_source (`kg.phases`)	def ingest_source(*, spec: Any, options: Any, uri: str, auth: (str,str), driver: Any, clients: ClientBundle, semantic_settings: Any, git_commit: str|None, reset_database: bool, execute_pipeline: Callable[..., str|None], ...) -> IngestionArtifacts	Core routine to ingest one document (`SourceSpec`). It executes the Neo4j GraphRAG pipeline for the source text (writing chunk nodes), then collects metadata and performs semantic enrichment if enabled [56] [57] . Steps: – Use `clients.splitter` in a scoped context (cache isolation per file) [57] [58] . – Generate a unique `ingest_run_key` for tagging this run's data [59] [60] . – **Execute pipeline:** calls `execute_pipeline(**kwargs)` which runs the upstream `SimpleKGPipeline` asynchronously to ingest chunks (returns a run ID) [58] [61] . – Retrieve chunk results: fetch cached chunks from the splitter (or run splitter if cache missed) [62] . – Build chunk metadata list (`ChunkMetadata` for each chunk) [63] [64] . – Call `ensure_document_relationships` to link Chunk nodes to a Document node in Neo4j (and update provenance properties) [65] [66] . – Package QA records: create `QaSourceRecord` with file info and list of `QaChunkRecord` (chunk IDs, text) [67] [68] . – Log basic stats (path, checksum, chunk count) and per-chunk entries [69] [70] . – If semantic enrichment is enabled, invoke it via `run_semantic_enrichment` (passing chunk data, LLM, etc.) to extract entities/relationships, and update semantic stats [71] [72] . – Return an `IngestionArtifacts` object with run_id, QA source record, log entry, chunk entries, and semantic stats [73] .	**Ne** no pip (in po `r` firs se `S` `r` [76] `sp` co co [58] `R` Ne ret

Module:Symbol	Signature (or Key)	Responsibility / Description	Sid
ChunkMetadata (`kg.pipeline`)	@dataclass ChunkMetadata(uid:str, sequence:int, index:int, checksum:str, relative_path:str, git_commit:str|None)	Metadata record for each text chunk ingested [82]. Captures unique `uid` (assigned by splitter), sequence number, original index, checksum of text, and source references (file path, commit) for provenance.	—
_build_chunk_metadata (`kg.pipeline` private)	def _build_chunk_metadata(chunks: Sequence[Any], *, relative_path:str, git_commit:str|None) -> list[ChunkMetadata]	Helper to convert a sequence of chunk objects into a list of `ChunkMetadata`. Ensures each chunk has a unique `uid`; computes a SHA-256 checksum of each chunk's text [85] [86]. Detects duplicate UIDs and raises error if found (should not happen unless splitter misbehaved) [86].	Pu sic da va
ensure_document_relationships (`db.neo4j_queries`)	def ensure_document_relationships(driver, *, database:str|None, source_path:Path, relative_path:str, git_commit:str|None, document_checksum:str, chunks_metadata: Sequence[ChunkMetadataLike]) -> None	Create or update a Document node for the source, and attach all ingested Chunk nodes to it in Neo4j [89] [90]. For each chunk in `chunks_metadata`, finds the corresponding `(:Chunk {uid})` node (inserted by the pipeline) and sets its `source_path`, `relative_path`, `git_commit`, `checksum`, and a stable `chunk_id` (first-time sequence number) if not already set [91] [92]. Ensures a `[:HAS_CHUNK]` relationship from Document to each Chunk [93] [94].	**Ne** qu Do so ch pr ev ch
IngestionArtifacts (`kg.phases`)	@dataclass IngestionArtifacts(run_id: str|None, qa_source: QaSourceRecord, log_entry: dict, chunk_entries: list[dict], semantic_stats: Any)	Container for the results of ingesting one source [98] [99]. Holds the pipeline run_id (if any), a QA source record (for later QA evaluation), a log entry summary, per-chunk entries, and semantic stats object.	—

Module:Symbol	Signature (or Key)	Responsibility / Description	Sid
perform_qa (`kg.phases`)	def perform_qa(*, driver, database:str|None, qa_sources: Sequence[QaSourceRecord], semantic_enabled: bool, semantic_totals: Any, qa_limits: Any, qa_report_dir: Path, qa_report_version: str, ...) -> QaOutcome	Executes the post-ingestion QA routine. It instantiates an `IngestionQaEvaluator` with all source records and thresholds, triggers the evaluation, and collates the results [102] [103] . Specifically: – Prepares a `QaThresholds` object from `qa_limits` (max allowed anomalies) [104] . – Builds a `SemanticSummary` object from `semantic_totals` stats [105] . – Creates the evaluator and calls `evaluate()` [106] . – Gathers the `qa_result` (pass/fail status, summary text, metrics, anomalies, etc.) and serializes it into a `qa_section` dict [107] [108] . – If QA *failed*, calls `rollback_ingest` to undo any ingested nodes for these sources and raises an error to halt the pipeline [109] [18] . – If passed, or after rollback, fetches graph node counts if not already in metrics [110] . – Returns a `QaOutcome` containing the QA section, final counts, and semantic summary [110] [111] .	**Ne** `d` co (do rel `c` rel `n` Als `r` (se wr Ma `q` `w` (in ex im `r`
QaSourceRecord (`qa.evaluator`)	@dataclass QaSourceRecord(path:str, relative_path:str, document_checksum:str, git_commit:str|None, chunks: list[QaChunkRecord], ingest_run_key:str|None)	Represents an ingested document in QA phase [118] . Contains file identity and a list of its chunks (as `QaChunkRecord` with uid, checksum, text) [119] [120] . Also carries the `ingest_run_key` to tie chunks to a specific ingest run.	—
QaLimits / QaThresholds (`kg.pipeline` / `qa.evaluator`)	@dataclass QaLimits (max_missing_embeddings:int=0, ...) in `kg.pipeline` *and* @dataclass QaThresholds (same fields) in `qa.evaluator`	These define the tolerances for ingestion anomalies. `QaLimits` is part of `PipelineOptions` (for CLI input) [123] [9] , and is converted to `QaThresholds` inside `perform_qa` for use in evaluator [104] [124] . All default to 0 (no anomalies allowed by default).	—

Module:Symbol	Signature (or Key)	Responsibility / Description	Sid
IngestionQaEvaluator (`qa.evaluator`)	class IngestionQaEvaluator(driver, database, sources, thresholds: QaThresholds, report_root:Path, report_version:str, semantic_summary: SemanticSummary|None)	Performs a series of Cypher checks and computes metrics after ingestion, comparing against thresholds. On `.evaluate()`, it calculates: – Graph counts (`documents`, `chunks`, relationships) [112]; – Missing embeddings count (chunks with no embedding vector) [113]; – Orphan chunks (chunks not linked to a Document) [113]; – Checksum mismatches (Chunk nodes whose stored checksum differs from expected) [128]; – If semantic data present: orphan semantic entities, etc., via `collect_semantic_counts` [129]. – Aggregates all metrics, determines pass/fail (`status`), and generates a detailed report (JSON and markdown) [130] [131].	**Ne** rea `d` de wr 11 ma `w` (no int `r` ret `Q`
rollback_ingest (`db.neo4j_queries`)	def rollback_ingest(driver, *, database:str|None, sources: Sequence[QaSourceRecordLike]) -> None	Removes graph data produced in a pipeline run, based on the `ingest_run_key` tags and chunk UIDs from the given sources [121] [134]. It executes Cypher queries to: – Delete all relationships with a matching `ingest_run_key` (for complete removal of those subgraphs) [135]. – Delete all nodes with a matching `ingest_run_key` that are *not* Document or Chunk (i.e., semantic nodes) [136]. – Delete all Chunk nodes whose UID is in the set of chunks for these sources [137]. – Delete any Document node by relative_path that has no remaining `:HAS_CHUNK` relationships (i.e., if all its chunks were removed) [138].	**Ne** de rel un ru Do fro Do or

Module:Symbol	Signature (or Key)	Responsibility / Description	Sic
DEFAULT_SCHEMA (`kg.pipeline`)	DEFAULT_SCHEMA = _load_default_schema() -> GraphSchema (loaded from `scripts/config/kg_schema.json`)	The default Neo4j graph schema definition (node/relationship types and properties) used by the pipeline's structured output. It's loaded at import time from a JSON file and validated into a `GraphSchema` object [139] [140]. This schema defines the structure of the knowledge graph (e.g., labels like Document, Chunk, and any semantic entity types).	Us Gr er inv sc co pr lab fo

FancyRAG's Custom Components (Clients, Splitter, Semantic LLM)

Module:Symbol	Signature / Definition	Responsibilities and Behavior
SharedOpenAIClient (`cli.openai_client`)	(Class in `cli.openai_client.py`) – not shown above; provides a singleton OpenAI API interface	**Shared HTTP client for OpenAI calls.** Manages base URL, API key, and common logic for both embedding and chat requests. Ensures all OpenAI requests (embeddings, chat completions) use a single `aiohttp` session and consistent error handling.
SharedOpenAIEmbedder (`kg.pipeline`)	class SharedOpenAIEmbedder(Embedder): **init**(client: SharedOpenAIClient, settings: OpenAISettings)	**Embedding vector generator using Shared client.** Implements the GraphRAG `Embedder` interface【14†L136- L143】[45]. On `embed_query(text:str)` it calls `client.embedding(text)` to get an embedding vector, then uses `ensure_embedding_dimensions` to verify or adjust the vector length [147] [148].

Module:Symbol	Signature / Definition	Responsibilities and Behavior
SharedOpenAILLM (`kg.pipeline`)	class SharedOpenAILLM(LLMInterface): **init**(client: SharedOpenAIClient, settings: OpenAISettings)	**Chat completion generator using Shared client.** Implements GraphRAG's `LLMInterface` for chat models [150] [151]. In `invoke(input, history, system_instruction)`, it builds a message list (prepends system and prior messages if provided, then user prompt) [152] [153], calls `client.chat_completion(messages, temperature, …)`, then extracts the model's text reply [154] [155]. Strips any Markdown fences from the output text (so that if the model returns a code block, it yields plain text) [155]. Raises `LLMGenerationError` if the API returns an error or empty content [156] [157].
CachingFixedSizeSplitter (`fancyrag.splitters`)	class CachingFixedSizeSplitter(FixedSizeSplitter)	**Chunk splitter with caching.** Extends the upstream `FixedSizeSplitter` (which splits text into fixed-size chunks) by adding result caching and UID stability [160] [161]. It maintains `_blueprints` (templates of chunk text/metadata for each input) and `_last_outputs` caches, and a `_scope_stack` for namespacing cache entries [162] [163]. – `scoped(scope_id)` context: push a scope (e.g. file path) to isolate cache keys within that context (pop on exit) [163] [164]. – `_cache_key(text)`: computes a cache key from input text or list of texts, prefixed by current scope if any [165] [166]. – `run(text, config=None)`: if `config` is provided, it delegates to base splitter's run; otherwise it computes a cache key and checks `_blueprints`. On a miss, calls base `run` to get chunks, stores a blueprint (list of {text,index,metadata}) and the output in caches. On a hit, it **rehydrates** new chunks with fresh `uid`s from the stored blueprint (ensuring identical chunk boundaries without re-running the split) [167] [168].

Module:Symbol	Signature / Definition	Responsibilities and Behavior
CachingSplitterConfig (`fancyrag.splitters`)	@dataclass CachingSplitterConfig(chunk_size:int, chunk_overlap:int=0)	Small config class to parametrize the caching splitter. Has a method `create_splitter()` that returns a new `CachingFixedSizeSplitter` with the given size and overlap [172] [173].
StructuredSemanticLLM (`kg.semantic_llm`)	class StructuredSemanticLLM(LLMInterface): **init**(client: SharedOpenAIClient, settings: OpenAISettings, *, schema: dict)	**LLM wrapper enforcing structured JSON output.** Extends GraphRAG's LLMInterface to generate model prompts that instruct the LLM to return a JSON adhering to a schema [177] [144]. In `invoke(input, ...)`: it builds the messages (similar to SharedOpenAILLM) and then prepares a `text_format` payload for the OpenAI API specifying the expected JSON schema (type `"json_schema"`, with a given `name` and the `schema` dict) [144] [178]. It calls `client.chat_completion(..., text={"format": {...}})`. If the model fails to comply (raising an OpenAI error about format), it falls back to a simpler format (`"json_object"`) and retries once [146] [179]. After getting a response, it extracts the raw JSON string from the result (removing any Markdown fences) and wraps it in an `LLMResponse` [180]. Raises `LLMGenerationError` if no content is produced [181] [180].

Module:Symbol	Signature / Definition	Responsibilities and Behavior
LLMEntityRelationExtractor (Upstream, used in FancyRAG)	class LLMEntityRelationExtractor(...): extract_for_chunk(schema:GraphSchema, prompt:str, chunk:TextChunk) -> Neo4jGraph (async)	*(This is an upstream component, but included here for completeness of semantic workflow.)* It takes a text chunk and uses an LLM to extract structured entities/relations from it. In FancyRAG, it is constructed with `llm=StructuredSemanticLLM`, `create_lexical_graph=False`, `on_error=OnError.RAISE`, and a concurrency limit [183] [185]. Its `extract_for_chunk` method will invoke the LLM to produce a `Neo4jGraph` object for that chunk. After extracting for all chunks (in parallel up to `max_concurrency`), it can `combine_chunk_graphs(None, chunk_graphs)` to merge individual chunk graphs into one graph [186]. It also provides `post_process_chunk(graph, chunk)` to attach the chunk text or metadata to the graph nodes (done after each extract) [187].

Config-Driven Pipeline (YAML & PipelineRunner)

(The following symbols are relevant when using the YAML configuration ingestion path.)

Module/ File:Symbol	Signature / Key	Responsibilities / Descrip
PipelineRunner (upstream)	`neo4j_graphrag.experimental.pipeline.config.PipelineRunner`	High-level runner that ex defined by a config file. F to run the `pipelines/kg_ingest` asynchronously [190]. It re instantiates all specified c manages their execution

Module/ File:Symbol	Signature / Key	Responsibilities / Descrip
kg_ingest.yaml (YAML config)	YAML defining components and their wiring for ingestion	Describes an equivalent p config: includes `neo4j_` user, password via env) [1] `llm_config` (class `Ope` API key from env and mo [194] , an `embedder_con` `OpenAIEmbeddings` wi `local-embedding-768` base_url from env) [195] , a `component_config` lis phase: – **splitter**: use `FixedSizeSplitter` w chunk_size=600, overlap= **embedder**: use `TextChu` that wraps the above em [197] ; – **extractor**: use `LLMEntityRelationEx` llm from llm_config, max_ [198] ; – **writer**: use `N` with Neo4j driver from ne database env) [199] ; – use `SinglePropertyExact` to merge identical nodes, database from env [200] . defines `connection_co` these: splitter → embedd chunks), embedder → ext chunks with embeddings writer (pass graph), write resolver_exact (final merg

Module/ File:Symbol	Signature / Key	Responsibilities / Descrip
Makefile `ingest`	`make ingest` target: `uv run python tools/run_pipeline.py $(f)`	Convenience entrypoint t pipeline. If an `f=<filep` provided, it passes that fi the pipeline; otherwise, it "Hello graph" string [1]

Call Graph of the FancyRAG Ingestion Workflow

Below is a simplified call flow for the code-driven pipeline (`kg_build.py` path). It illustrates how control and data move from the CLI down to Neo4j writes and back to final QA:

```
flowchart TD
    subgraph CLI
        A["scripts/kg_build.py"]:::script -->|args|
B["fancyrag.cli.kg_build_main.main()"]:::func
    end
    subgraph FancyRAG Pipeline (Code)
        B --> C["PipelineOptions ← parse CLI args"]:::data
        B --> D["run_pipeline(options)"]:::func
        D --> D1["resolve_settings(...) -> ResolvedSettings"]:::func
        D --> D2["discover_sources(...) -> SourceSpec(s)"]:::func
        D --> D3["build_clients(...) -> ClientBundle"]:::func
        D --> D4{For each SourceSpec}
        D4 --> E["ingest_source(spec, clients, ...) ->
IngestionArtifacts"]:::func
        E -.->|reset_database True (first file)| F[[Neo4j: MATCH (n) DETACH
DELETE n]]:::neo4j
        E --> G["execute_pipeline(..., splitter, embedder, llm) ->
run_id"]:::func
        subgraph Upstream SimpleKGPipeline
            G1["splitter.run(text) -> TextChunks"]:::func -->
G2["embedder.embed_query(chunk.text) -> vector"]:::func -->
G3["writer.write(chunk, vector) -> (:Chunk node)"]:::neo4j
            G2 --> G4["llm (not used in base ingest)"]
```

```
            G3 --> G5[/"... for each chunk"/]
            G5 --> G6["return run_id (commit)"]:::data
        end
        E <-- G6 --- G
        E --> H["splitter.get_cached(text) -> TextChunks"]:::func
        E --> I["build_chunk_metadata(chunks) -> list[ChunkMetadata]"]:::func
        E --> J["ensure_document_relationships(Document, chunks)"]:::neo4j
        E --> K["QaSourceRecord + QaChunkRecords"]:::data
        E --> L["run_semantic_enrichment(...) ->
SemanticEnrichmentStats"]:::func
        subgraph Semantic Enrichment
            L1["for chunk in chunks:
LLMEntityRelationExtractor.extract_for_chunk(schema, chunk) ->
Neo4jGraph"]:::func --> L2[/"await all (max_concurrency)"/]
            L2 --> L3["combine_chunk_graphs(chunk_graphs) -> Neo4jGraph"]:::func
            L3 --> L4["annotate_semantic_graph(graph, metadata)"]:::func
            L4 --> L5["SanitizingNeo4jWriter.run(graph) -> (:Entity nodes,
rels)"]:::neo4j
            L2 --> L6["if failure and configured, write failure artifact
(json)"]:::file
        end
        E --> M["return IngestionArtifacts(run_id, qa_source, log_entry, chunks,
semantic_stats)"]:::data
        D4 -->|loop| N["...next SourceSpec..."]:::ellipsis
        D --> O["perform_qa(all qa_sources, qa_limits) -> QaOutcome"]:::func
        O --> O1["IngestionQaEvaluator.evaluate()"]:::func
        subgraph QA Evaluation (IngestionQaEvaluator)
            Q1["collect_counts() ->
{'documents':d,'chunks':c,'relationships':r}"]:::neo4j
            Q2["count_missing_embeddings(chunk_uids)"]:::neo4j
            Q3["count_orphan_chunks(chunk_uids)"]:::neo4j
            Q4["count_checksum_mismatches(chunk_uid↦checksum)"]:::neo4j
            Q5["if semantic_enabled: collect_semantic_counts()"]:::neo4j
            Q6["compile metrics & compare to QaThresholds"]:::func
            Q7["write_ingestion_report(markdown+json)"]:::file
        end
        O1 -.->|metrics| P["qa_result (pass/fail, anomalies, report)"]:::data
        O -->|if fail| R[[Neo4j: rollback_ingest(ingest_run_key) → DELETE
ingested nodes]]:::neo4j
        O --> S["QaOutcome(qa_section, counts, semantic_summary)"]:::data
        D --> T["Assemble final log dict (status, counts, run_ids, ... QA,
semantic)"]:::data
        D --> U["write log JSON to file & print"]:::file
    end
    classDef func fill:#e7f7ff,stroke:#89b3f8;
    classDef data fill:#fdfdfd,stroke:#ccc;
    classDef neo4j fill:#ffe7d6,stroke:#f8a789;
```

```
    classDef file fill:#f7f7c0,stroke:#f0e68c;
    classDef script fill:#dfe7fd,stroke:#7f93e0;
    classDef ellipsis fill:#bbb,stroke:#bbb;
```

Figure: Call flow for FancyRAG's code-based ingestion. The CLI parses arguments into `PipelineOptions`, then `run_pipeline` orchestrates discovery, client setup, and processing of each source file. The **upstream** SimpleKGPipeline (within `execute_pipeline`) handles chunking, embedding, and writing chunks to Neo4j. FancyRAG then uses the same splitter output to link chunks to a Document node and performs optional semantic extraction (via the LLM-based extractor and a second Neo4j write). Finally, an ingestion QA evaluation runs Cypher queries to detect any anomalies, potentially rolling back if thresholds are exceeded. The result is a JSON log of the entire run, including any QA report.

Configuration Surfaces (Environment Variables & YAML Keys)

FancyRAG's ingestion pipeline is configured via both environment variables and YAML keys. The table below summarizes these config surfaces, their default or expected values, any validation, and where they are used in the pipeline:

Env Variable / YAML Key	Default (if any)	Validation / Constraints	Where
NEO4J_URI `neo4j_config.params_.uri`	(no default – must be provided)	Must use bolt/neo4j scheme with host:port; validated at startup [205].	FancyR for driv Pipeline
NEO4J_USERNAME `neo4j_config.params_.user`	(no default – must be provided)	(No format validation beyond non-empty.)	FancyR (auth fo Pipeline
NEO4J_PASSWORD `neo4j_config.params_.password`	(no default – must be provided)	(No format validation; handled as secret.)	FancyR (auth fo Pipeline
NEO4J_DATABASE `neo4j_config.params_.(none)` ¹	Default: *None* (use Neo4j's default DB)	If provided, must be a valid database name (no explicit code validation).	FancyR `Neo4j` driver a `drive` `databa` writer a target t
NEO4J_BOLT_ADVERTISED_ADDRESS, **NEO4J_HTTP_ADVERTISED_ADDRESS**	Default: *None*	Must be valid addresses if provided (validated by URL parse for HTTP) [216] [217].	(Not dir these a relevan monito

Env Variable / YAML Key	Default (if any)	Validation / Constraints	Where
OPENAI_API_KEY `llm_config.params_.api_key`	(no default – must be provided for OpenAI usage)	Not validated in code (but required if OpenAI is used) [218] .	FancyR `OpenA` calls [21] config s OpenAI
OPENAI_PROJECT `llm_config.params_.project`	(no default; optional)	No validation. (Likely used for Azure OpenAI "deployment" name or tracking.)	FancyR doesn't into Op use if n
OPENAI_MODEL	Default: `"gpt-5-mini"`	Must be one of allowed model names (guarded against unsupported values) [225] [226] . If a fallback model is used, requires `OPENAI_ENABLE_FALLBACK=true` .	FancyR `OpenA` `gpt-5` model) explicitl ignorin older m
OPENAI_EMBEDDING_MODEL **EMBEDDING_MODEL**	Default: `"text-embedding-3-small"`	(No strict validation beyond allowed set in code – defaults to above if not provided) [228] . These two env keys are synonyms: either can specify the embedding model name.	FancyR `OpenA` set, falls `small"` YAML: s embedd drawn f
OPENAI_EMBEDDING_DIMENSIONS **EMBEDDING_DIMENSIONS**	Default: *1536* (in code, from default model)	If set, must be a positive integer; code raises error if non-positive or not an int [231] [232] . These override the expected dimensionality of embeddings.	FancyR by env, var), `Op` embedd `make` to creat [235] . YA embedd model c
OPENAI_MAX_ATTEMPTS	Default: `3`	Parsed as int >0; errors out if not a positive integer [236] [237] .	FancyR `OpenA` for Ope `Share` request above, YAML: r their ov

Env Variable / YAML Key	Default (if any)	Validation / Constraints	Where
OPENAI_BACKOFF_SECONDS	Default: `0.5`	Parsed as float >0; must be >0 or error [234] [239] .	FancyR `OpenA` betwee retry lo
OPENAI_TEMPERATURE	Default: `0.3`	Parsed as float ≥0; error if negative [240] [241] .	FancyR `OpenA` respons `Struc` random LLM co determ
OPENAI_ENABLE_FALLBACK	Default: `true`	Parsed as boolean (accepts “true/false” etc.); if false, code forbids using fallback model [244] [245] .	FancyR must re model i automa `gpt-5` applica
OPENAI_BASE_URL / **OPENAI_ALLOW_INSECURE_BASE_URL**	Default: *None* / `False`	If `OPENAI_BASE_URL` is set (to use an alternate API endpoint, e.g., Azure or local proxy), it must be a valid http(s) URL; if the scheme is not https and `ALLOW_INSECURE` is not “true”, validation throws an error [247] [248] .	FancyR `OpenA` the clie of api.o `ALLOW` http (no used fo OpenAI config (OpenAI
EMBEDDING_API_BASE_URL / **EMBEDDING_API_KEY**	(no default; required for local embedding service)	Base URL must be a valid http(s) URL (similar validation rules as OpenAI_BASE_URL) [247] [251] . API key has no format constraint.	FancyR an exte `OpenA` and `em` [253] . In likely ro this key `requi` EMBED [254] [255] via env `OpenA` For exa suggest 20010.

Env Variable / YAML Key	Default (if any)	Validation / Constraints	Where
QDRANT_URL / **QDRANT_API_KEY**	*None* (optional; legacy integration)	URL must be a valid http(s) URL if provided [216] [217] . API key no constraints.	FancyR loads a **Howev activel** vector s appear optiona this env or simil used in
QDRANT_NEO4J_ID_PROPERTY_NEO4J / **..._EXTERNAL**	Default: `"chunk_id"` for both	(No validation; expects string property names.)	FancyR link to N tell the use as t into `Qd` etc. [252] itself.
INDEX_NAME (env)	Default: `"text_embeddings"` (if not set)	(No validation in code)	Used in E.g., the `text_` for vect used in embedd EMBED
FULLTEXT_INDEX_NAME, **FULLTEXT_LABEL**, **FULLTEXT_PROPERTY** (env)	Defaults: `"chunk_text_fulltext"`, `"Chunk"`, `"text"`	(No validation)	Used by `create` above) to mana [259] . No related chunks.

<small>¹ The YAML `neo4j_config` has no explicit `database` field – instead, the database is fed via env (`NEO4J_DATABASE`) directly into the writer and resolver components [215] .</small>

Upstream Touchpoints and Stability

FancyRAG's ingestion pipeline builds atop the **Neo4j GraphRAG (graphRAG)** library. The table below identifies where FancyRAG calls into this upstream library, the specific symbols used, and notes on their stability and possible need for replacement or wrapping:

FancyRAG Use (Location)	Upstream Symbol & Version
`_execute_pipeline` **uses** > `SimpleKGPipeline.run_async` 260 261	`neo4j_graphrag.experimental.pipeline.kg_builder.SimpleKGPipeline`
`run_semantic_enrichment` **uses** > `LLMEntityRelationExtractor` (async) 183 143	`neo4j_graphrag.experimental.components.entity_relation_extractor.`

FancyRAG Use (Location)	Upstream Symbol & Version
Custom writer extends `Neo4jWriter` 76 263	`neo4j_graphrag.experimental.components.kg_writer.Neo4jWriter` (and

FancyRAG Use (Location)	Upstream Symbol & Version
Schema loading uses > `GraphSchema` and `Neo4jGraph` 140 266	`neo4j_graphrag.experimental.components.schema.GraphSchema` > `neo4j_graphrag.experimental.components.types.Neo4jGraph`

FancyRAG Use (Location)	Upstream Symbol & Version
Splitter usage of > `FixedSizeSplitter` 267 268	`neo4j_graphrag.experimental.components.text_splitters.FixedSizeSp`
OpenAI API classes (replaced upstream `OpenAIEmbeddings` / `OpenAILLM`)	`neo4j_graphrag.embeddings.openai.OpenAIEmbeddings` `neo4j_grap`

FancyRAG Use (Location)	Upstream Symbol & Version
PipelineRunner & Config (YAML ingestion path)	`neo4j_graphrag.experimental.pipeline.config.PipelineRunner`
Exact match resolver (YAML only)	`neo4j_graphrag.experimental.components.resolver.SinglePropertyExa`

Upstream Version: FancyRAG is built against **Neo4j GraphRAG v1.12.0** [270] . All upstream references above correspond to that version (on the default branch, likely **main**). Many critical components are still marked experimental in GraphRAG, indicating potential breaking changes in future releases. FancyRAG mitigates this by pinning the version and wrapping where necessary. The decision matrix below addresses how to handle these dependencies going forward.

Initial Decision Matrix for Components

This table summarizes the plan for each major component in the FancyRAG ingestion pipeline – whether to **Keep** as-is, **Wrap** with an abstraction, **Adapt** (modify/enhance), **Replace**, or **Split** (break into separate parts) – with rationale:

Component	Action	Rationale	Evidence
Pipeline Entrypoint (YAML vs Code)	**REPLACE** YAML pipeline with Code pipeline (make code canonical)	Having two parallel ingestion paths caused drift [5]. The code-based pipeline (`kg_build.py`) includes newer defaults (GPT-5 model, QA gating) and better integrates with the MCP server, whereas the YAML path was a stop-gap. Plan: update documentation and Makefile to use `kg_build.py` for ingestion [7], and treat the YAML config as legacy. This simplifies maintenance and ensures one source of truth for ingestion.	[5] [7]
Core Ingestion Pipeline (SimpleKGPipeline usage)	**WRAP/ KEEP**	Continue using upstream `SimpleKGPipeline` within `_execute_pipeline`, but keep it abstracted behind FancyRAG's interface. The upstream pipeline reliably handles chunk creation, vector embedding, and initial writes, so we benefit from its robustness. By wrapping it (as done in `_execute_pipeline`), FancyRAG can insert custom logic (caching, error handling) around it [260] [262]. In the future, if GraphRAG pipeline APIs change or if a custom implementation is needed, we can swap out `_execute_pipeline` internals without affecting higher-level code. For now, no need to reimplement chunk ingestion from scratch.	[260] [145]
OpenAI Integration (LLM & Embeddings)	**KEEP (Custom)**	FancyRAG's custom OpenAI client classes (SharedOpenAIEmbedder/LLM) should be retained. They already *replaced* the upstream OpenAIEmbeddings/LLM to allow a shared client and advanced usage (structured output, caching) [45] [146]. This has improved efficiency and capability. The upstream equivalents are stable but feature-limited; our versions offer better control. We will maintain our custom integration and update it as needed (e.g., if OpenAI API changes).	[45] [146]

Component	Action	Rationale	Evidence
Text Splitting	**KEEP + ADAPT**	Continue using fixed-size chunking, with our caching wrapper. The caching `FixedSizeSplitter` has proven valuable in avoiding duplicate work [171]. We will keep this strategy. If upstream GraphRAG introduces new splitting modes (e.g., by semantics or sections), we might **adapt** by adding new splitter implementations or extending ours. But the fixed-size approach (600 chars) is working and aligns with project needs (and profile presets). No immediate change required beyond tuning chunk size defaults as needed.	[27] [171]
Neo4j Writing	**KEEP + WRAP**	Retain use of GraphRAG's Neo4jWriter, with our Sanitizing wrapper. We've already **adapted** the writer by subclassing to handle complex properties [264] [76]. This ensures stability if upstream changes property handling: our wrapper can intercept any new issues. Neo4j writing itself doesn't need replacement – leveraging the optimized writer in GraphRAG is beneficial. We'll continue wrapping it to inject any needed logic (e.g., setting ingest_run_key, sanitization).	[265] [76]
Semantic Extraction	**KEEP** (GraphRAG extractor)	Continue using the upstream LLMEntityRelationExtractor with our StructuredSemanticLLM. This combination is providing structured semantic graphs effectively. While experimental, it encapsulates complex prompt engineering and graph assembly that would be costly to reinvent. We monitor its stability; if it changes (or if we need a different approach), we might adapt or configure it differently. But for now, keeping it as is – integrated with our custom LLM – is the best approach for semantic enrichment.	[143] [144]

Component	Action	Rationale	Evidence
Post-Processing Duplicate Entities	**SPLIT/ ADAPT** (Future)	Currently, the YAML pipeline merges identical entities via `SinglePropertyExactMatchResolver`, but the code path does not [200]. We acknowledge this gap and will address it by either adding a similar deduplication step in code or enforcing uniqueness through constraints. This might be implemented as a **split** responsibility – e.g., have a separate post-ingestion routine (or an extension of QA) to find and merge duplicate nodes. Adapting the code path to handle this will bring parity with YAML. Until then, this is a known difference; in practice, it may not be critical if our schema or extraction tends to avoid exact duplicates.	[200] [271]
QA Gating	**KEEP** (FancyRAG QA)	The ingestion QA mechanism (missing embeddings, orphan chunks, etc.) is a custom enhancement in FancyRAG – there is no equivalent in upstream GraphRAG. We will keep this in place, as it provides valuable quality control. As data scales, we might **adapt** threshold values or extend checks (e.g., content-specific validations), but the framework (IngestionQaEvaluator and rollback) remains. This is a distinct competitive advantage of FancyRAG's pipeline, ensuring data integrity in the KG.	[5] [109]
GraphRAG Library Version	**ADAPT/ WRAP** (Upgrade strategy)	As GraphRAG evolves beyond v1.12.0, we'll need to adapt. The plan is to pin the current version (ensuring stability), and test upgrades in isolation. We use wrappers (as above) around most experimental parts, which will cushion breaking changes. If a future GraphRAG release promotes some experimental APIs to stable (or changes them), we can update our calls in one place. In short, continue wrapping key touchpoints and adapt our integration when upgrading library versions, rather than exposing unstable calls throughout code.	[270] [269]

HEAD Commit & Branch: This report is based on **FancyRAG commit** `0ff7b46717da4310b1626b0756551b7bd175d81b` (likely on the default **main** branch as of 2026-01-11) and upstream **neo4j-graphrag-python v1.12.0** (commit `b8408185ca0df6cc141a29d5d28ad006c9844c5b` on main, tagged as 1.12.0) [270] [272]. All code references and decisions above correspond to those versions. Going forward, any library upgrades will be assessed with the above matrix in mind, to ensure FancyRAG's ingestion pipeline remains robust and maintainable.

1 235 259 Makefile
https://github.com/DrJLabs/FancyRAG/blob/0ff7b46717da4310b1626b0756551b7bd175d81b/Makefile

2 190 191 192 run_pipeline.py
https://github.com/DrJLabs/FancyRAG/blob/0ff7b46717da4310b1626b0756551b7bd175d81b/tools/run_pipeline.py

3 6 7 204 2026-01-10-makefile-ingest.md
https://github.com/DrJLabs/FancyRAG/blob/0ff7b46717da4310b1626b0756551b7bd175d81b/docs/issues/2026-01-10-makefile-ingest.md

4 10 11 13 19 28 29 116 125 126 127 174 kg_build_main.py
https://github.com/DrJLabs/FancyRAG/blob/0ff7b46717da4310b1626b0756551b7bd175d81b/src/fancyrag/cli/kg_build_main.py

5 269 271 codebase-evaluation.md
https://github.com/DrJLabs/FancyRAG/blob/0ff7b46717da4310b1626b0756551b7bd175d81b/docs/codebase-evaluation.md

8 9 12 14 15 16 17 20 25 26 27 36 37 38 40 45 46 47 48 49 50 54 55 74 75 76 77 78 79 80 82 85 86 87 88 100 101 114 115 123 139 140 141 142 143 145 147 148 149 150 151 152 153 154 155 156 157 158 159 169 182 183 184 185 186 187 188 189 260 261 262 263 264 265 pipeline.py
https://github.com/DrJLabs/FancyRAG/blob/0ff7b46717da4310b1626b0756551b7bd175d81b/src/fancyrag/kg/pipeline.py

18 21 22 23 24 30 31 32 33 34 35 39 41 42 43 44 51 52 53 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 81 83 84 98 99 102 103 104 105 106 107 108 109 110 111 117 132 133 phases.py
https://github.com/DrJLabs/FancyRAG/blob/0ff7b46717da4310b1626b0756551b7bd175d81b/src/fancyrag/kg/phases.py

89 90 91 92 93 94 95 96 97 121 134 135 136 137 138 211 neo4j_queries.py
https://github.com/DrJLabs/FancyRAG/blob/0ff7b46717da4310b1626b0756551b7bd175d81b/src/fancyrag/db/neo4j_queries.py

112 113 118 119 120 122 124 128 129 130 131 evaluator.py
https://github.com/DrJLabs/FancyRAG/blob/0ff7b46717da4310b1626b0756551b7bd175d81b/src/fancyrag/qa/evaluator.py

144 146 177 178 179 180 181 242 semantic_llm.py
https://github.com/DrJLabs/FancyRAG/blob/0ff7b46717da4310b1626b0756551b7bd175d81b/src/fancyrag/kg/semantic_llm.py

160 161 162 163 164 165 166 167 168 170 171 172 173 175 176 267 268 caching_fixed_size.py
https://github.com/DrJLabs/FancyRAG/blob/0ff7b46717da4310b1626b0756551b7bd175d81b/src/fancyrag/splitters/caching_fixed_size.py

193 194 195 196 197 198 199 200 201 202 203 208 210 212 213 215 220 221 222 223 224 230 243 256 kg_ingest.yaml
https://github.com/DrJLabs/FancyRAG/blob/0ff7b46717da4310b1626b0756551b7bd175d81b/pipelines/kg_ingest.yaml

205 206 207 209 214 216 217 218 219 225 226 227 228 229 231 232 233 234 236 237 238 239 240 241 244 245 246 247 248 249 250 251 252 253 254 255 257 258 settings.py
https://github.com/DrJLabs/FancyRAG/blob/0ff7b46717da4310b1626b0756551b7bd175d81b/src/config/settings.py

266 structured_output.py
https://github.com/DrJLabs/FancyRAG/blob/0ff7b46717da4310b1626b0756551b7bd175d81b/src/fancyrag/kg/structured_output.py

270 requirements.lock
https://github.com/DrJLabs/FancyRAG/blob/0ff7b46717da4310b1626b0756551b7bd175d81b/requirements.lock

272 Releases · neo4j/neo4j-graphrag-python · GitHub
https://github.com/neo4j/neo4j-graphrag-python/releases